ARAŞTIRMA SORULARI

1.Metot neden kullanılır:

-Kod tekrarını önler:
–Aynı işlemi yapan kodları defalarca yazmak yerine tek bir metotta toplayıp her yerde çağırabilirsiniz.

-Okunabilirliği artırır:
–Binlerce satırlık karmaşık kodları küçük,anlamlı parçalara(alt programlara)bölerek kodun anlaşılmasını kolaylaştırır.

-Hataları azaltır ve bakımı kolaylaştırır:
–Yapılması gereken bir değişiklik veya hata düzeltmesi varsa,bunu kodun her yerinden değil,sadece ilgili metodun içinde tek seferde yaparsınız.

-Test etmeyi kolaylaştırır:
–Küçük parçalara bölünmüş kod bloklarının doğruluğunu ve testini yapmak çok daha pratiktir.

-Modülerlik sağlar:
–"Bir kere yaz,her yerde çağır"mantığıyla programın daha profesyonel ve yapısal olmasını sağlar.

2.Parametre ve dönüş değeri hangi ihtiyacı çözer?:

Parametreler,bir fonksiyona çalışması için ihtiyaç duyduğu dinamik verileri dışarıdan aktarma ihtiyacını çözer.

-Esneklik ve tekrar kullanabilirlik:
–Bir fonksiyona her seferinde farklı veriler göndererek aynı işlemi tekrarlatabilirsiniz.Örneğin;kareAl(sayi)fonksiyonunda sayi bir parametredir.Bu sayede fonksiyon sadece 5'in değil,gönderdiğiniz her sayının karesini alabilir.

-Bağımlılığı azaltma:
–Fonksiyonun kendi dışındaki global değişkenlere bağımlı kalmasını önler.Fonksiyon,ihtiyacı olan her şeyi parametre olarak eline alır.

2.Dönüş değeri(return)hangi ihtiyacı çözer?(çıktı ihtiyacı):
Dönüş değeri,bir fonksiyonun yaptığı işlemin sonucunu kodun geri kalanında kullanabilme ihtiyacını çözer.

-Süreklilik ve iş akışı:
–Fonksiyonda üretilen verinin kaybolmamasını sağlar.Örneğin;fiyatHesapla()fonksiyonundan dönen sonucu alıp,bir sonraki adımda vergiEkle()fonksiyonuna besleyebilirsiniz.

-Ekrana yazdırmak yerine veriyi elde etmek:
–Bir fonksiyon sonucu ekrana yazdırırsa(print),o veri sadece ekranda görünür ve kod içinde tekrar kullanılmaz.Dönüş değeri(return)ise o veriyi bilgisayarın hafızasında tutarak üzerinde işlem yapmanıza olanak tanır.

3.Class ve object arasındaki ilişki nedir?

-Class(sınıf):
–Bir nesnenin sahip olacağı özellikleri(değişkenler/alanlar) ve yapabileceği işleri(metotlar/davranışlar)belirleyen soyut bir şablon,plan veya taslaktır.Bellekte doğrudan bir veri tutmaz,sadece bir kural bütünüdür.

-Object(nesne):
–sınıf şablonundan türetilen,bellekte somut bir yer kaplayan ve kendine ait verileri barındıran gerçek bir örnektir(instance).

4.Bir class'ın tek bir sorumluluğa sahip olması ne demektir?

O sınıfın yalnızca tek bir iş yapması ve değişmek için tek bir sebebinin bulunması gerektiği anlamına gelir.

TEMEL MANTIĞI:

-Tek görev:
–Sınıf yalnızca belirli bir alandaki işlemleri üstlenir.Örneğin,bir kullanıcı sınıfı hem veritabanı kaydı yapmamalı hem de e-posta göndermemelidir.

-Net tanım:
–Sınıfa baktığıızda "Bu sınıf ne işe yarıyor?"diye sorduğunuzda tek ve net bir cevap alabilmesiniz(örneğin:"Sadece kullanıcının şifresini doğrular").

Kötü ve iyi örnek

-Kötü(çok sorumluluklu):kullanıcı adında bir sınıf hem kullanıcının adını/mailini tutuyor,hem veritabanına bağlanıp SQL sorgusu çalıştırıyor hem de PDF faturası oluşturup mail atıyorsa,bu sınıfın birden fazla sorumluluğu vardır.

İyi(tek sorumluluklu):

-kullanıcı:
–Sadece kullanıcı verilerini(modelini)tutar.

-KullaniciRepository:
–Sadece veritabanı okuma/yazma işlemlerini yapar.

-FaturaServisi:
–Sadece fatura oluşturma ve gönderme işlemlerini yapar.

Sağladığı Avantajlar:

-Kolay bakım:
–Bir yerde değişiklik yapıldığında diğer tarafların bozulma riski azalır.

-Okunabilirlik:
–Kod daha küçük,sade ve anlaşılır parçalara bölünür.

-Test edilebilrilik:
–Küçük ve odaklı sınıfları birim testlerle(unit test)kontrol etmek çok daha kolaydır.

5.Encapsulation ve abstraction kavramlarını temel seviyede araştırın.

ENCAPSULATİON(kapsülleme):
–Bir sınıfa ait verilerin(özelliklerin)ve bu veriler üzerinde işlem yapan kodların(metotların)bir arada tutulması ve dış dünyadan gizlenmesidir.

Verilere dışarıdan doğrudan erişilmesini engeller.Bunun yerine,verilere güvenli bi şekilde erişmek ve onları değiştirmek için özel köprüler(Getter ve Setter memtotları)kullanılır.

-Gerçek hayat benzetmesi:
–Bir ilaç kapsülü.Kapsülün içindeki kimyasal tozları(verileri)doğrudan göremez veya dokunamazsınız.Kapsül(sınıf)koruyucu bir çeper görevi görür ve içindekileri dış etkilerden korur.

-Temel amaç:
–Veri güvenliğini sağlamak ve hatalı veri girişini önlemek(Data Hiding).

ABSTRACTİON(soyutlama):
–Karmaşık detayları gizleyerek sadece kullanıcının ihtiyacı olan temel özellikleri gösterme sürecidir.

Bir sistemin nasıl çalıştığıyla değil,ne yaptığıyla ilgilenir.Arka plandaki karmaşık kod yapıları veya algoritmalar gizlenir;kullanıcıya sadece basit bir arayüz(ınterface)sunulur.

-Gerçek hayat benzetmesi:
–Bir araba sürmek.Arabayı hızlandırmak için sadece gaz pedalına basarsınız.Pedala bastığınızda motorun içindeki pistonların nasıl çalıştığını,yakıtın nasıl ateşlendiği gibi detayları bilmenize gerek yoktur.Pedal,sizin için bir soyutlama katmandır.

-Temel amaç:
–Karmaşıklığı azaltmak ve kodun yeniden kullanılabilirliği artırmak.

6.Her şeyi main metoduna yazmanın ne gibi sorunları olabilir?

1.Okunabilirlik ve anlaşabilirlik azalır:
-Kod yığınları:
–Binlerce satırdan oluşan tek bir metodu yukarıdan aşağıya okumak ve mantığı takip etmek imkansız hale gelir.

-Bilişsel yük:
–Kodu inceleyen bir yazılımcı(veya birkaç ay sonra siz),küçük bir kısmı anlamak için tüm kodun akışını tutmak zorunda kalır.

2.Bakım ve geliştirme zorlaşır(maintainability):
-Değişiklik riski:
–Kodun bir yerinde yapılan küçük bir değişklik,aynı metodun altındaki bambaşka ve tamamen alakasız bir öantığı kolayca bozabilir.

-Hata bulma(debugging):
–Bir hata(bug) oluştuğunda,hatanın kaynağını binlerce satır arasından tespit etmek çok uzun zaman alır.

3.Kod tekrarı artar(code duplication)
-Yeniden kullanılamama:Projenin farklı yerlerinde aynı işlemi(örneğin veri tabanına bağlanma veya bir yazıyı formatlama)yapmanız gerektiğinde,kodu kopyalayıp yapıştırmak zorunda kalırsınız.

-DRY(Don’t Repeat yourself)ihlali:
–Aynı mantığı 5 farklı yere yapıştırdıysanız,bir kural değiştiğinde 5 farklı yeri de

güncellemeniz gerekir.Birini unutursanız sistemde tutarsızlık başlar.

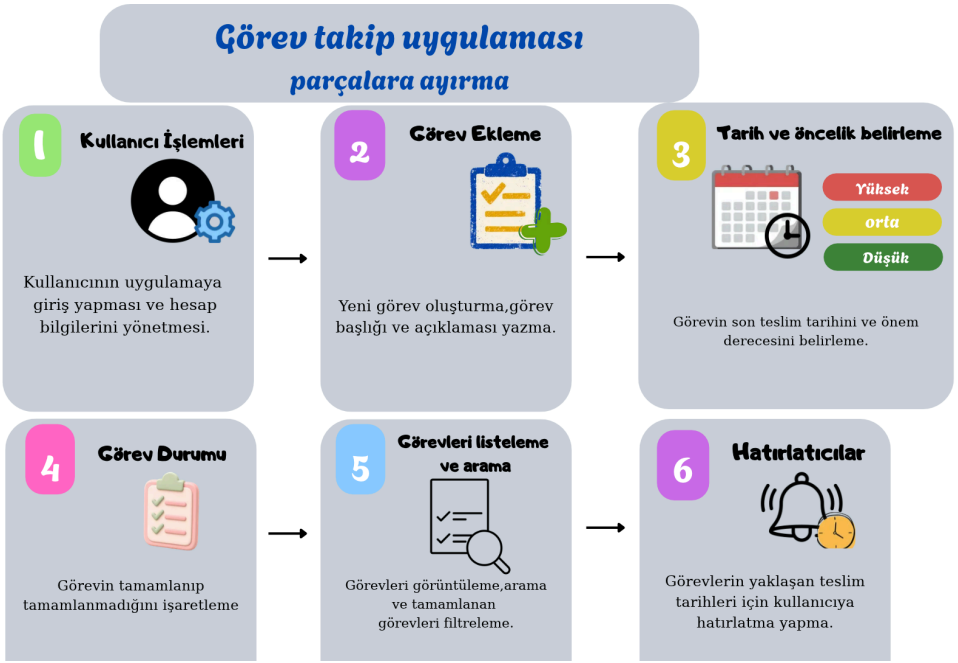
4.Test edilemez hale gelir(testability):
-Birim testi(unit test)imkansızlığı:
–Modern yazılım geliştirmede fonksiyonlar tek tek test edilir.Her şey main içindeyse,sistemin sadece belirli bir parçasını(örneğin "KDV hesaplama"mantığını) otomatik test araçlarıyla tek başına test edemezsiniz.

5.Esneklik ve ölçeklenebilirlik kaybolur:
-Nesne yönelimli programlama(OOP) devre dışı kalır:
–Sınıflar(classes),arayüzler(ıntrefaces)ve kapsülleme(Encapsulation)gibi yazılım mimarisini güçlü kılan hiçbir prensibi kullanamazsınız.

-Geliştirici iş birliği zorlaşır:
–Aynı proje üzerinde çalışan birden fazla yazılımcı varsa,herkes aynı main metodunu içeren dosyayı değiştirmeye çalışacağından,versiyon kontrol sistemlerinde(Git gibi)sürekli kod çakışmaları(marge conflict)yaşanır.

GÜN SONUNDA TESLİM EDECEĞİNİZ NOT

1.Bir görev takip uygulamasını kod yazmadan parçalara ayırın.



2.Hangi bilgilerin tutulacağını,hangi işlemlerin yapılacağını ve bunların nasıl gruplanabileceğini yazın.

Görev Takip uygulaması:

1.Hangi bilgiler tutulacak?
-Kullanıcı adı ve şifre
-Görev başlığı ve açıklaması
-Görevin oluşturulma tarihi
-Son teslim tarihi
-Görevin öncelik durumu(Yüksek,orta,düşük)
-Görevin tamamlanıp tamamlanmadığını

2.Hangi işlemler yapılacak?
-Yeni görev ekleme
-Görevleri listeleme
-Görevleri güncelleme
-Görevleri tamamlandı olarak işaretleme
-Görev arama ve filtreleme
-Teslim tarihi yaklaşan görevler için hatırlatma yapma

3.Bunlar nasıl gruplanabilir?
-Kullanıcı İşlemleri:
–Giriş yapma ve hesap bilgileri yönetme.

-Görev Yönetimi:
–Görev ekleme,silme ve güncelleme.

-Tarih ve Öncelik Yönetimi:
–Teslim tarihi ve öncelik belirleme.

-Görev Durumu:
–Görevleri tamamlandı veya devam ediyor olarak işaretleme.

-Listeleme ve Arama:
–Görevleri görüntüleme,arama ve fltreleme.

-Hatırlatıcılar:
–Yaklaşan görev tarihlerini bildirme.

3.En az üç metot ve en az bir class önerin;neden var olduklarını açıklayın.

1.Class(sınıf):
-Görev sınıfı:
–Uygulamadaki görevlerin bilgilerini tutmak için oluşturulur.

Tutacağı bilgiler:
-Görev adı
-Görev açıklaması
-Son teslim tarihi
-Görev durumu

Neden var?
–Her görevin bilgilerini düzenli bir şekilde saklamak ve görevleri birbirinden ayırmak için kullanılır.

2.Metotlar:
-GörevEkle():
–Yeni bir görev oluşturur ve görev listesine ekler.

Neden var?
–Kullanıcının yapacağı yeni işleri uygulamaya kaydetmesini sağlar.

-GörevSil():
–Seçilen görevi listeden kaldırır.

Neden var?
–Artık yapılmayacak veya yanlışıkla veya yanlışıkla eklenmiş görevleri silmek için kullanılır.

-GörevleriListele():
–Kayıtlı bütün görevleri ekranda gösterir.

Neden var?
–Kullanıcının yapması gereken işleri kolayca görmesini sağlar.